

Lanka Ride — one design pattern per member

Group 2026-Y2-S1-MLB-B13G2-05. Each person owns one pattern inside the code they already own. Screens, security rules, and the saved data stay the same.

A design pattern is a named way to arrange a small piece of code so the next change has one obvious place to go. The patterns below were chosen because the existing behaviour already had that shape. They were not added as empty classes.

Spring already keeps each `@Service` as one shared object. That is why the Singleton in this project is not a second copy of a service. It is the maintenance rule book, which really is one stateless object. The Multiton is the three report periods, which really are three shared objects.

Member	ID	Owns	Pattern
Sakalasooriya S.A.A.A.	IT25102266	Vehicle management	Builder
De Silva D.L.K.C.	IT25102264	Fleet maintenance	Singleton
Samaranayake P.I.S.	IT24101349	Booking	State
Kavindi P.D.N.	IT25102265	Payment	Strategy
Wickramasinghe R.D.W.K.G.S.	IT24101008	Dashboard and reports	Multiton
Pahasara Y.A.P.	IT24102871	Support and notifications	Observer

1. Sakalasooriya S.A.A.A. — Builder

Code: `vehicle/VehicleBuilder.java`, used by `VehicleService.update`.

Why this pattern. A vehicle has registration, category, brand, model, seats, gearbox, fuel, features, photo, price, deposit, status, and location. Writing that list of setters in the service hides the idea “copy the checked form onto the saved car.” A builder names that copy. A Singleton would not help, because each car is a different row. A Factory would only help if there were several kinds of vehicle class. There is one `Vehicle` entity.

How it works. `VehicleBuilder.from(incoming)` reads the form vehicle one field at a time and returns the builder. `applyTo(existing)` writes those fields onto the row that is already in the database. The branch is still set by the service after it has loaded the branch, so the builder does not guess a branch. Validation in `normalize` still runs first.

What a viva answer should say. “I used a Builder because a car has many fields and the update should copy them in one place. The screens and the database columns did not change.”

2. De Silva D.L.K.C. — Singleton

Code: `fleet/MaintenanceRules.java`, used by `MaintenanceService.update` and `MaintenanceService.close`.

Why this pattern. The rules are the same for every maintenance job: a closed record cannot be edited, closing an already closed record does nothing, and the edit form cannot mark a job closed (closing has its own action). Those rules have no fields that change per request, so one object is enough. The constructor is private. Every caller uses `MaintenanceRules.getInstance()`. A Multiton would fit if each status needed its own object with different data. Here the rules are one book, so a Singleton is the smaller pattern.

How it works. The single instance is created when the class loads (`private static final`). `requireEditable` throws if the status is CLOSED. `isClosed` lets `close` return the same record when it is already closed. `statusAfterEdit` turns a CLOSED choice on the edit form into IN_PROGRESS, which is what the service did before.

What a viva answer should say. "I used a Singleton because the maintenance rules are one shared policy, not one policy per record. Spring services are also singletons, but this class is the explicit rule book."

3. Samaranayake P.I.S. — State

Code: `booking/BookingFlow.java`, used by `BookingService.approve`, `deny`, and `cancel`.

Why this pattern. A booking moves through PENDING, APPROVED, DENIED, ONGOING, COMPLETED, and CANCELLED. Approve and deny are legal only from PENDING. Cancel is legal from PENDING or APPROVED. Putting those answers on the status itself is the State pattern: the current state decides what is allowed. A chain of `if` statements in the service hides which status owns which action.

How it works. `BookingFlow` is an enum with one constant per `BookingStatus`. Each constant stores whether approve, deny, and cancel are allowed. `BookingFlow.of(booking.getStatus())` picks the state. `requireApprove`, `requireDeny`, and `requireCancel` throw the same messages as before when the move is illegal. The service still checks the vehicle, the date overlap, and who is allowed to cancel.

What a viva answer should say. "I used State because a booking's status decides which buttons are legal. The date checks stay in the service. Payment still moves an approved booking to ONGOING after a successful charge."

4. Kavindi P.D.N. — Strategy

Code: `payment/PaymentGateway.java` and `payment/DummyGateway.java`. `PaymentController` depends on the interface.

Why this pattern. Checkout needs “charge this card and tell me if it was declined.” That job can be done by the sandbox gateway today and by a bank gateway later. The controller should not name `DummyGateway`. Strategy means the caller holds an interface, and one class supplies the real steps. A Singleton gateway would not leave room for a second gateway.

How it works. `PaymentGateway.charge` returns a small decision: declined or not, card brand, and the last four digits. `DummyGateway` implements that interface and still checks the Luhn number, the brand, the expiry, the name, and the security code. A Visa ending in 0002 or 0000 is still declined. Full card numbers are still not stored.

What a viva answer should say. “I used Strategy so the payment screen talks to a gateway interface. `DummyGateway` is the strategy we run in the project. Swapping the class does not change the booking or the invoice.”

5. Wickramasinghe R.D.W.K.G.S. — Multiton

Code: `dashboard/ReportWindow.java`, used by `ReportService.buildReport`.

Why this pattern. Reports are daily, monthly, or annual. Those three windows do not change, and every request for “monthly” needs the same rule. A Multiton keeps one shared object per name. `ReportWindow.of("monthly")` always returns the same monthly object. Unknown or missing names return the daily object, which is what the old switch did. A Singleton would be wrong because there are three periods, not one. A new object on every request would also work, but it would throw away the fact that the set is fixed.

How it works. The three objects are created once, and the constructor is private. `resolve` turns a reference date into a start, an end, a label, and the period name. Daily is that calendar day. Monthly is the first of the month up to the first of the next month. Annual is 1 January up to the next 1 January. Totals, branch rows, and the CSV still come from `ReportService`.

What a viva answer should say. “I used a Multiton because there are exactly three report periods and each name should return the same window object. The money is still read from the payment rows. The report does not store a second copy of the totals.”

6. Pahasara Y.A.P. — Observer

Code: `support/NotificationObserver.java`, `support/LoggingNotificationObserver.java`, and `support/NotificationService.java`.

Why this pattern. Saving a notification and reacting to it are different jobs. The service writes the log row. Observers hear that the row was saved. The logger is the observer in this project. A later mail sender or SMS sender can implement `NotificationObserver` and be registered by Spring, without a change to the save method. Strategy would fit if the service picked one channel algorithm. Here more than one reaction can listen, so Observer is the better fit.

How it works. `send` still builds a `NotificationLog`, marks it SENT when the recipient is present and FAILED when it is blank, and saves it. It then calls `onNotified` on every observer.

`LoggingNotificationObserver` writes the same NOTIFY line as before. `email` still calls `send` with the EMAIL channel. Retry of failed rows is unchanged.

What a viva answer should say. "I used Observer so the notification log does not know who wants to hear about a new row. The logger is one observer. Ticket create and ticket reply still call the notification service after they save."

What was left alone

No table, screen, or security rule was added for these patterns. Vehicle search, transfer, retire, and delete stay in `VehicleService`. Maintenance dates still cannot be in the past. Booking approval still checks the vehicle and overlapping dates. Payment is still an append-only ledger. Report numbers are still calculated from the payment and booking rows when the page opens.